Certified minimum numbers of matches for four heat-recovery benchmark instances

chemistry-open-problems · 25 October 2026 · solved (preliminary) for four instances, partial result for the others

Solver: Claude Opus 5.5 (effort xhigh), sub-agent of the orchestrator, in two sessions (24 and 25 October 2026) · Independent verification: pending

Abstract. Letsios, Kouyialis and Misener (2018) published 48 benchmark instances of the minimum number of matches problem, the NP-hard mixed-integer linear program at the core of sequential heat exchanger network synthesis, and observed that state-of-the-art solvers cannot solve moderately sized instances to global optimality. We determine the exact optimum of four instances that were left without a proof of optimality: `37sp-yfyv` = 36, `20sp1` = 19, `23sp1` = 22 (best value known before: 23) and `unbalanced10` = 25. Every upper bound is an explicit network, checked in exact rational arithmetic by two independently written programs, the second one against the original model of Letsios et al. with their own big-M values. The lower bounds of the first three instances follow from a block lemma (every connected component of the match graph is heat balanced and cascade feasible) and an exhaustive exact enumeration of blocks; the same argument raises the lower bound of the four 33-stream `balanced15` instances from 31 to 32. The lower bound for `unbalanced10` is a VIPR certificate (0.54 GB) of a pure LP-based branch-and-bound proof, written by exact-arithmetic SCIP 10.0.2 for a strengthened transshipment model; every row of that model is proved valid here and re-derived independently from the data, and the certificate is accepted both by VIPR's checker and by a second checker written from the format specification. On the way we found that VIPR's reference checker accepts two kinds of invalid inference steps (rounding over continuous variables, branching disjunctions with a fractional right-hand side); we give a patch and test certificates.

The question

The source is Letsios, Kouyialis and Misener [1] (arXiv v2, Sec. 9.2, p. 30), whose data, code and solver logs are public:

> *"Observe that state-of-the-art approaches cannot, in general, solve moderately-sized problems with 30-40 streams to global optimality. For example, none of the test cases in the Grossmann (2017) or large scale test sets is solved to global optimality within the specified time limit."*

The same section states that each run used "absolute gap 0.99, relative gap 4%". The solver logs of the repository show that `unbalanced10` (value 25) and `37sp-yfyv` (value 36) are among the instances reported as solved, but every one of their six runs (CPLEX 12.6.3 and Gurobi 6.5.2, three formulations) stopped at the 4 % tolerance with best bound 24, respectively 35. For `20sp1` and `23sp1` the time limit was reached (best bounds 16 and 17). Twenty-five instances of the benchmark had no matching upper and lower bound.

Theorem 1. On the data files of the benchmark of Letsios, Kouyialis and Misener [1], the minimum number of matches is

`37sp-yfyv`: 36, `20sp1`: 19, `23sp1`: 22, `unbalanced10`: 25.

Moreover the minimum number of matches is at least 32 for `balanced15` and `balanced15_random0/1/2` (previous bound 31), at least 32 for `unbalanced15`, at least 23 for `22sp1`, and at least 26 for `balanced12` and `balanced12_random0/1/2`. The optimum of the transshipment MILP of Letsios, Kouyialis and Misener [1], with any of the three big-M files of their repository, has the same value as the minimum number of matches (Proposition 4).

instance	N	LKM18 UB / LB	here UB / LB	status
`37sp-yfyv`	38	36 / 35	36 / 36	optimal 36 (block lemma)
`20sp1`	21	19 / 16	19 / 19	optimal 19 (block lemma)
`23sp1`	24	23 / 17	22 / 22	optimal 22 (block lemma), new UB
`unbalanced10`	23	25 / 24	25 / 25	optimal 25 (VIPR certificate)
`unbalanced15`	33	36 / 35	36 / 32	open
`22sp1`	24	25 / 23	25 / 23	open
`b12` (4 instances)	27	28–29 / 26–27	28–29 / 26	open
`b15` (4 instances)	33	35 / 31	35 / 32	open, new LB
`u17` (4 instances)	41	43 / 39–40	43 / –	open
`u20` (4 instances)	47	50–51 / 45	50–51 / –	open
`large_scale0/1/2`	162	175–239 / 123–135	–	open, not attempted

Table 2 / Status of the 25 instances without matching bounds in 2018. LKM18: best upper / lower bound over the six runs in the repository of Letsios, Kouyialis and Misener [1] (floating-point solver output, lower bounds rounded up). Here: exact network (UB) and Lemma 6 or a VIPR certificate (LB). Families: `b12` stands for `balanced12` and `balanced12_random0/1/2`, likewise `b15`, `u17` (`unbalanced17`), `u20`.

For `balanced12_random1` the certified bound 26 is below the floating-point bound 27 of the 2018 runs; no certified lower bound in Table 2 contradicts a floating-point one.

1 The problem

An instance (file `data/mip_instances/<set>/<name>.dat` of the repository of Letsios, Kouyialis and Misener [1]) has hot streams $i \in H$ ($|H| = n$, hot utilities included with the loads fixed by the minimum utility cost step), cold streams $j \in C$ ($|C| = m$, cold utilities included) and temperature intervals $t = 0, \dots, k-1$, interval 0 being the hottest. $Q^H_{it} \geq 0$ is the heat that hot stream i supplies in interval t and $Q^C_{jt} \geq 0$ the heat that cold stream j needs in interval t. All numbers are read as exact decimal rationals. We write $H_i[a..b] = \sum_{a \leq t \leq b} Q^H_{it}$, $H_i(\leq t) = H_i[0..t]$, $H_i = H_i(\leq k-1)$, and $D_j[a..b]$, $D_j(\leq t)$, D_j likewise; $N = n + m$; $R_t = \sum_{s<t} (\sum_i Q^H_{is} - \sum_j Q^C_{js})$ is the heat cascade. For all 54 data files we checked exactly that $R_k = 0$, $R_t \geq 0$ for all t, that the residuals printed in the file equal R_t, and that every stream has positive total heat.

Definition 3. A *feasible network* is a family of numbers $q_{ijt} \geq 0$ such that $\sum_i q_{ijt} = Q^C_{jt}$ for all j, t and $r_{it} := H_i(\leq t) - \sum_{s \leq t} \sum_j q_{ijs} \geq 0$ for all i, t, with $r_{i,k-1} = 0$. Its *match set* is $E(q) = \{(i, j) : \sum_t q_{ijt} > 0\}$, and the *minimum number of matches* is

$\min_q |E(q)|$.

Heat of hot stream i produced in interval s can thus be used in interval s or in any colder interval (the minimum approach temperature is built into the intervals); r_{it} is the heat of i carried from interval t to interval $t+1$. The transshipment MILP of Papoulias and Grossmann [2], as implemented by Letsios, Kouyialis and Misener [1] (`lib/exact_methods/transshipment_mip_solver.py`), reads: minimise $\sum_{ij} y_{ij}$ subject to $\sum_j q_{ijt} + \rho_{i,t+1} = Q^H_{it} + \rho_{it}$, $\sum_i q_{ijt} = Q^C_{jt}$, $\sum_t q_{ijt} \le U_{ij} y_{ij}$, $\sum_i \rho_{i0} = \sum_i \rho_{ik} = 0$, $y \in \{0,1\}$, $q, \rho \ge 0$, with big-M values U_{ij} read from the files `<name>.bgm` of the directories `simple`, `gundersen` and `greedy` under `data/bigM_parameters/<set>/`.

Proposition 4. For each instance of Theorem 1 and each of the three big-M files, the optimum of this MILP equals the minimum number of matches.

Proof. If (y, q, ρ) is feasible for the MILP, then $\rho_{i,t+1} = r_{it}$, so q is a feasible network and $E(q) \subseteq \{(i,j) : y_{ij} = 1\}$; hence the MILP optimum is at least the minimum number of matches, whatever the U_{ij}. Conversely, for each network of Section 2, setting $y_{ij} = 1$ exactly on $E(q)$ and $\rho_{i,t+1} = r_{it}$ gives a feasible point: the script `check_network_lkm18.py` checks in exact arithmetic every constraint of the MILP, including $\sum_t q_{ijt} \le U_{ij} y_{ij}$ for the three big-M files, and that the objective equals the stated number of matches. With the lower bounds of Sections 3 and 4 this gives equality. □

2 Upper bounds: explicit networks

The directory `certificates/solutions/` contains, for 22 instances, a network given by rational heats q_{ijt} (JSON). They were found with floating-point solvers and with Lemma 7 below, then rounded to exact flows by an integer max-flow computation (all data scaled by the least common multiple of the decimal denominators). Two programs check them:

- `check_solutions.py` (with `hen_exact.py`) checks $q \ge 0$, every cold balance, $r_{it} \ge 0$, $r_{i,k-1} = 0$, and that every declared match carries positive heat;
- `check_network_lkm18.py`, written separately with its own parsers, checks the network as a point of the MILP of Letsios, Kouyialis and Misener [1] with y set to the indicator of $E(q)$ (Proposition 4).

Both print `ALL PASS` for the 22 networks, which have 36, 19, 22, 25 matches for `37sp-yfyv`, `20sp1`, `23sp1`, `unbalanced10`, and reproduce the best 2018 values for the other 18 instances of Table 2 (they improve none of them).

3 The block lemma

Definition 5. A nonempty set $K \subseteq H \cup C$ of streams is a *block* if $\sum_{i \in K \cap H} H_i = \sum_{j \in K \cap C} D_j$ and $\sum_{i \in K \cap H} H_i(\le t) \ge \sum_{j \in K \cap C} D_j(\le t)$ for every t. Let $c_{\max}$ be the largest number of parts of a partition of $H \cup C$ into blocks.

Lemma 6. Every feasible network q satisfies $|E(q)| \ge N - c_{\max}$.

Proof. Let G be the bipartite graph on $H \cup C$ with edge set $E(q)$. Every stream has positive total heat, so it has an incident edge; every connected component K of G therefore has at least $|K| - 1 \geq 1$ edges. All heat of a hot stream of K goes to its neighbours, which lie in K, and all demand of a cold stream of K comes from its neighbours, so $\sum_{i \in K} H_i = \sum_{i,j \in K,t} q_{ijt} = \sum_{j \in K} D_j$, and for every t

$$\sum_{j \in K} D_j(\leq t) = \sum_{j \in K} \sum_{s \leq t} \sum_{i \in K} q_{ijs} = \sum_{i \in K} (H_i(\leq t) - r_{it}) \leq \sum_{i \in K} H_i(\leq t).$$

Hence K is a block, the $c(G)$ components form a partition into blocks, $c(G) \leq c_{\max}$, and $|E(q)| \geq \sum_K (|K| - 1) = N - c(G) \geq N - c_{\max}$. □

Lemma 7. If $c_{\max} = 2$ and $|E(q)| = N - 2$, then G is a forest with two components A, B, $\{A, B\}$ is a partition into two blocks, and the restriction of q to A (to B) is a feasible network of the sub-instance A (B) whose match graph is a spanning tree.

Proof. In the proof of Lemma 6, $N - 2 = |E| \geq N - c(G)$ forces $c(G) = 2$ and exactly $|K| - 1$ edges per component. □

Lemma 7 only guides the search for networks; it is not used in any proof of a bound.

Computation of $c_{\max}$ (`block_bound.py`). With L the least common multiple of the decimal denominators, stream v gets the integer weight $w_v = LH_i$ (hot) or $-LD_j$ (cold) and an integer cumulative profile. All zero-weight subsets are enumerated exactly by meet in the middle (all subset sums of the two halves, exact hash join); a zero-weight subset is a block if and only if the sum of the two half-profiles is nonnegative in every interval (all partial sums are below 2^{62}, asserted). A 2-block partition exists iff some block A containing stream 0, $A \neq H \cup C$, has a block as complement. A partition into three blocks exists iff there are blocks $A \ni 0$ and B such that $X = (H \cup C) \backslash A$ is a block, $B \subsetneq X$ contains the lowest-index stream of X, and $X \backslash B$ is a block (unions and differences of nested blocks are blocks, since both defining conditions are additive); the search over all such pairs is exhaustive. A partition into four or more blocks yields one into three by merging, so "no 3-block partition" means $c_{\max} \leq 2$.

instance	N	zero-sum subsets	blocks	2-part.	$N - c_{\max}$
`37sp-yfyv`	38	4 833	2 431	14	36
`20sp1`	21	15	12	4	19
`23sp1`	24	17	12	3	22
`22sp1`	24	1	1	0	23
`balanced12`, `_random0/1/2`	27	$\approx 6 \cdot 10^4$	$\approx 1.8 \cdot 10^4$	0	26
`balanced15`	33	1 817 217	518 054	0	32
`balanced15_random0/1/2`	33	1.75–$3.42 \cdot 10^6$	0.63–$1.00 \cdot 10^6$	0	32
`unbalanced15`	33	470 203	94 888	0	32

Table 8 / Exhaustive exact enumeration (`run_block_bounds.sh`, 41 s in total). "2-part.": number of partitions into two blocks; no instance has a partition into three blocks.

With the networks of Section 2 this proves the values 36, 19 and 22 of Theorem 1. For `unbalanced10` ($N = 23$) the block lemma gives at most 22, so a different argument is needed.

4 unbalanced10: a strengthened model and a VIPR certificate

4.1 The strengthened transshipment model

Let $\mathrm{rc}_i(t) = \min\big(R_{t+1}, H_i(\le t)\big)$ for $0 \le t \le k-2$ and $\mathrm{rc}_i(-1) = \mathrm{rc}_i(k-1) = 0$.

Lemma 9. In every feasible network, $0 \le r_{it} \le \mathrm{rc}_i(t)$.

Proof. Summing the definition of r_{it} over i and using the cold balances gives $\sum_i r_{it} = \sum_i H_i(\le t) - \sum_j D_j(\le t) = R_{t+1}$; all $r_{i't}$ are nonnegative, so $r_{it} \le R_{t+1}$, and $r_{it} \le H_i(\le t)$ by definition. □

Given a feasible network q, put $y_{ij} = 1$ if $(i,j) \in E(q)$ and $y_{ij} = 0$ otherwise.

Proposition 10. (q, r, y) satisfies each of the following constraints. Consequently, if the MILP formed by them together with $\sum y \le K$ has no solution, every feasible network has at least $K+1$ matches.

(1) Balances: $\sum_j q_{ijt} + r_{it} - r_{i,t-1} = Q^H_{it}$ and $\sum_i q_{ijt} = Q^C_{jt}$ (with $r_{i,-1} = r_{i,k-1} = 0$).

(2) Bounds: $0 \le r_{it} \le \mathrm{rc}_i(t)$ and $0 \le q_{ijt} \le u_{ijt} := \min\big(Q^C_{jt}, Q^H_{it} + \mathrm{rc}_i(t-1)\big)$.

(3) Disaggregated links: $q_{ijt} \le u_{ijt} y_{ij}$.

(4) Aggregated links: $\sum_t q_{ijt} \le U_{ij} y_{ij}$, where U_{ij} is the minimum over all labellings $\lambda : \{0, \dots, k-1\} \to \{\mathrm{in}, \mathrm{out}\}$ of $\mathrm{cap}(\lambda) = \sum_{\lambda(u)=\mathrm{in}} Q^C_{ju} + \sum_{\lambda(u)=\mathrm{out}} Q^H_{iu} + \sum_{\lambda(u)=\mathrm{in},\lambda(u+1)=\mathrm{out}} \mathrm{rc}_i(u)$.

(5) Cold windows ($Q^C_{jT} > 0$, $s \le T$):

$$\sum_i \min\big(D_j[s..T],\ \mathrm{rc}_i(s-1) + H_i[s..T]\big) y_{ij} \ge D_j[s..T].$$

(6) Hot windows ($Q^H_{is} > 0$, $s \le T$, right-hand side positive):

$$\sum_j \min\big(H_i[s..T], D_j[s..T]\big) y_{ij} \ge H_i[s..T] - \mathrm{rc}_i(T).$$

(7) Partner counts: for each window row $\sum_e a_e y_e \ge b$, $\sum_e y_e \ge p$, where p is the least number of coefficients, taken largest first, whose sum reaches b.

Proof. (1) are the definitions. (2) r: Lemma 9. q: the heat that i delivers in interval t is at most $r_{i,t-1} + Q^H_{it}$ and the heat that j receives there is Q^C_{jt}. (3) is (2) when $y_{ij} = 1$, and $q_{ijt} = 0$ when $y_{ij} = 0$. (4) Fix a labelling λ. Intervals labelled in contribute $q_{iju} \le Q^C_{ju}$. On a maximal run $[a..b]$ of out intervals, the heat i delivers to anybody is $r_{i,a-1} + H_i[a..b] - r_{ib} \le \mathrm{rc}_i(a-1) + H_i[a..b]$, and $\mathrm{rc}_i(a-1)$ is paid by the in-to-out transition at $a-1$ (or vanishes if $a = 0$). So $\sum_t q_{ijt} \le \mathrm{cap}(\lambda)$, and $= 0$ if $y_{ij} = 0$. (5) $D_j[s..T] = \sum_i \sum_{s \le t \le T} q_{ijt}$, and the i-th inner sum is at most $D_j[s..T]$, at most the heat i delivers in the window, $r_{i,s-1} + H_i[s..T] - r_{iT} \le \mathrm{rc}_i(s-1) + H_i[s..T]$, and zero if $y_{ij} = 0$. (6) Decompose the flow along the chain of hot stream i into amounts $g(a \to t) \ge 0$, $a \le t$, of heat produced in a and delivered in t ($\sum_t g(a \to t) = Q^H_{ia}$, $\sum_a g(a \to t) = \sum_j q_{ijt}$, e.g. first in, first out), and split each delivery among the cold streams in proportion to q_{ijt}. Let O_j be the heat produced in $[s..T]$ and delivered to

j inside $[s..T]$: $O_j \le \min(H_i[s..T], D_j[s..T])\, y_{ij}$. Heat produced in $[s..T]$ and delivered after T is part of r_{iT}, so $\sum_j O_j \ge H_i[s..T] - r_{iT} \ge H_i[s..T] - \mathrm{rc}_i(T)$. (7) With binary y, a knapsack row $\sum a_e y_e \ge b$, $a_e \ge 0$, needs at least p variables equal to one. Finally, pairs with $U_{ij} = 0$ and variables with $u_{ijt} = 0$ or $\mathrm{rc}_i(t) = 0$ are omitted from the model; they are zero in every feasible network by (2) and (4). □

The minimum in (4) is computed by a two-state dynamic programme (`hen_cuts.py`). It is the capacity of a minimum cut in the two-stream network source $\to \mathrm{hot}_u$ (Q^H_{iu}), $\mathrm{hot}_u \to \mathrm{hot}_{u+1}$ ($\mathrm{rc}_i(u)$), $\mathrm{hot}_u \to \mathrm{cold}_u$ (∞), $\mathrm{cold}_u \to$ sink (Q^C_{ju}), but validity only uses the direct argument above.

4.2 The certified computation

The model of Proposition 10 for `unbalanced10`, with objective $\sum y$ and the extra row `maxm`: $\sum y \le 24$, is the CPLEX-LP file `certificates/vipr/unbalanced10_le24.lp` (122 binary, 816 + 147 continuous variables, 2453 rows of which 1247 are window and partner-count rows; exact decimal coefficients; SHA-256 `e3dcb597...`). Its certification has five steps, each printing PASS or FAIL (`verify_vipr.sh`):

(1) the LP file is regenerated from the data by `hen_lpfile.py` and must be byte-identical;

(2) `check_lp_model.py` re-derives every row and bound of the LP file from the data with separately written code (exact cascade recomputed from the data, U_{ij} checked as the capacity of an explicitly reconstructed labelling, window coefficients, partner counts, the row `maxm`), checks that every omitted variable is forced to zero, and that the file has no other bounds or binaries;

(3) `check_vipr_model.py` checks that every constraint of the certificate's problem section equals, up to a positive factor, a row or a variable bound of the LP file, that every integer variable of the certificate is a binary of the LP file, and that the relation to prove is `infeas`; the certificate therefore proves the infeasibility of a relaxation of the LP-file MILP;

(4) `viprchk` (VIPR, commit 30f2951, with the two patches of Section 5) verifies the certificate in exact GMP rational arithmetic, on the raw file and on the output of `viprcomp`;

(5) `vipr_check_independent.py`, a second checker written from the format specification [3] [4] without using `viprchk`'s code, verifies the same file.

The certificate was produced by SCIP 10.0.2 with SoPlex 8.0.2 [5] [6], built from source with exact rational solving, run with `exact/enable = TRUE`, presolving, cutting planes and conflict analysis switched off (`certificates/vipr/scip_exact_safe.set`), single thread. SCIP reports infeasibility after 10 233 nodes (5 117 infeasible leaves, maximal depth 32; 287 s on an idle machine, 553 s with other jobs running). The certificate (VIPR 1.0 syntax) has 540 587 890 bytes and SHA-256

`17967e67e607271c2fc60fc77a185da2badcbb84459d7deaeaddd5a4601ec352`;

it has 1 085 variables (122 integer), 4 623 problem constraints (2 453 rows and 2 170 variable bounds) and 191 697 derivation steps: 10 232 assumptions, 170 552 linear combinations (14.8 million multipliers in all), 5 797 roundings and 5 116 unsplitting (branching) steps; there is no "weak" or "incomplete" step. It is not stored in the repository: `run_vipr_certificate.sh` regenerates it (the same 10 233 nodes were obtained on 24

and 25 October with two different builds); its size and hash, the solver log and the verification log are in `certificates/vipr/`.

step	check	result
1	LP file regenerated from the data, byte-identical	PASS
2	every row and bound re-derived from the data	PASS
3	certificate problem $\subseteq$ LP model, RTP `infeas`	PASS
4a	patched `viprchk`, raw certificate	PASS (68 s)
4b	`viprcomp`, then patched `viprchk`	PASS (64 s)
5	`vipr_check_independent.py`	PASS (199 s)

The whole of `verify_vipr.sh` took 7 min 40 s. The certificate is also accepted by the unpatched `viprchk` and by the build with the canonicalisation patch only, so it uses neither of the patterns of Section 5. As negative tests, `vipr_mutation_tests.sh` changes the certificate in four places (right-hand side of $\sum y \le 24$ replaced by 25, so that the problem becomes feasible; one multiplier changed; the two assumptions of an unsplitting step swapped; a rounding step claiming $y \ge 2$); both checkers reject each of the four corrupted files. Together with the 25-match network of Section 2 this proves `unbalanced10` $= 25$.

Trusted base. The proofs of Lemma 9 and Proposition 10; `check_lp_model.py` and `check_vipr_model.py`; and either `viprchk` with its patches and GMP, or the second checker (Python with gmpy2); the C++ and Python tool chains. SCIP, SoPlex, `viprcomp` and the LP generator are not trusted.

5 Checking the checkers

Two gaps in VIPR's reference checker. Reading `viprchk.cpp` (commit 30f2951; the current master, commit 99eec19 of 24 October 2026, has the same code) we found two inference rules that are checked too weakly:

(1) a rounding step (`rnd`) requires integer coefficients on the integer variables but does not reject nonzero coefficients on continuous variables, although Chvátal–Gomory rounding is only valid when the whole support is integer;

(2) an unsplitting step (`uns`) checks that the two branching constraints are $ax \le \beta$ and $ax \ge \beta + 1$ with a integral on integer variables, but not that β is an integer; for fractional β the disjunction misses integer points.

The certificates `certificates/vipr/viprchk_tests/bad_rnd_continuous.vipr` ("proves" that $x \ge \frac{1}{2}$, $x \le \frac{1}{2}$, x continuous, is infeasible) and `bad_uns_fractional.vipr` ("proves" that $x \ge 1$, $x \le 1$, x integer, is infeasible by branching on $x \le \frac{1}{2} \mid x \ge \frac{3}{2}$) are accepted by the unpatched `viprchk`. The patch `certificates/viprchk_harden.patch` adds both checks, and also bounds checks on constraint indices (the original performs them only in debug builds); the patched checker and `vipr_check_independent.py` reject both test certificates, and still accept the example certificates of the VIPR distribution that the original accepts. The hardening patch only adds rejections.

Canonicalisation patch. GMP's C++ stream input does not canonicalise rationals ("5/5" is stored as 5/5, and GMP's equality test assumes canonical operands), while SCIP

prints non-canonical fractions. The patch `certificates/viprchk_canonicalize.patch` canonicalises every rational after it is read (four places); canonicalisation does not change any value.

Earlier SCIP certificates that failed. With cutting planes at all nodes, or at the root only, SCIP 10.0.2 wrote Gomory mixed-integer cut derivations that `viprcomp` and `viprchk` could not complete and verify ("Failed to derive constraint . . . _gom. . . "). With conflict analysis on, a certificate of 2.19 GB contained 9 828 "weak" conflict rows such as `AggrRow_5376`, which claims that 116 of the 122 variables sum to at most 18, derived from $\sum y \leq 24$ with the instruction to complete it with *global* bounds; the row is valid only under the local fixings of the six missing variables to one, so the derivation as written is wrong and the verification failed at that row. We therefore use no cutting planes and no conflict analysis; with these settings the certificate contains no weak derivation at all.

Floating-point pitfalls. The 36-match network returned by floating-point SCIP for `37sp-yfyv` is infeasible in exact arithmetic (deficit exactly 0.1 heat units), and every one of the six match lists of the 2018 repository for this instance needs 37 pairs exactly after exact pruning (`side_findings.py`); an exact 36-match network exists nonetheless (two spanning trees on a 2-block partition, Lemma 7). The solution writer of the 2018 code prints $y = 1$ for every $y > 10^{-10}$, so its match lists may contain pairs that carry negligible heat; we always recompute flows exactly.

6 Novelty, other instances, limitations

Novelty check (25 October 2026). The 13 works citing Letsios, Kouyialis and Misener [1] in OpenAlex (and the 11 in Semantic Scholar) were screened by title and abstract; the doctoral thesis of Kouyialis [7] contains the 2018 study with the same values, and the survey Letsios et al. [8] reports no new values. The repositories of the 2018 study have no commit after February 2019; a GitHub search on the instance names and key phrases found only those repositories and a student replication without bound tables; the AI-driven campaigns listed in the protocol of this repository (GitHub users `ipitchford` and `mmislan`) have no heat-exchanger-network entry. To our knowledge, none of the four optimal values had been proved before. Furman and Sahinidis [9] (read in full) report CPLEX 7.0 values 19, 25, 23 and 36 for `20sp1`, `22sp1`, `23sp1` and `37sp-yfyv`, all marked as not converged after 7 h, and give no lower bounds; the value 22 for `23sp1` is below the best value reported there and in 2018 (23). Cerdá, Cafaro and Cafaro [10] (read in full) treat a different problem (detailed network structures with stream splitting, 45 literature examples) and report no value for the data files used here. Chen, Grossmann and Miller [11], the origin of the balanced and unbalanced instances, was not read in full.

Other instances. We also tried the strengthened model on the open instances with the smallest gaps. Floating-point SCIP 10.0.2 with its default settings (cutting planes on), 1800 s per run, did not decide $\sum y \leq 35$ for `unbalanced15` (final dual bound 33.9), $\sum y \leq 24$ for `22sp1` (21.5), or $\sum y \leq 27$ for `balanced12`, `_random0` and `_random2` (≤ 28 for `_random1`; dual bounds 24.7–25.0). Without cutting planes, as required here for a verifiable certificate, the root LP bounds of the strengthened model are 28.9 for

`unbalanced15` (36 needed), 18.2 for `22sp1` (25 needed) and 22.2 for `balanced12` (28 needed), against 21.1 (25 needed) for `unbalanced10`, which already took 10 233 nodes; we did not run the certified pipeline on them. These statements are exploration, not claims.

Limitations. The data files are taken as given (decimal numbers read as exact); the underlying stream temperatures and the minimum utility cost step were not re-derived. The lower bound for `unbalanced10` rests on software (`viprchk` or our second checker) and on the model-match scripts; it has not been re-checked by an independent verifier yet.

7 Reproduction

All scripts are in `scripts/` and print PASS/FAIL; times are wall-clock on WSL2 (Fedora 44), one core of a desktop CPU.

script	checks	time
`check_solutions.py`	22 networks, exact	2 s
`check_network_lkm18.py`	22 networks as points of the 2018 MILP	2 s
`run_block_bounds.sh`	Lemma 6 bounds, exact enumeration	41 s
`side_findings.py`	floating-point findings for `37sp-yfyv`	17 s
`run_vipr_certificate.sh`	produces the certificate (exact SCIP)	5–10 min
`verify_vipr.sh`	steps 1–5 of Section 4	7.7 min
`vipr_mutation_tests.sh`	four corrupted certificates rejected	6 min

References

[1] Dimitrios Letsios, Georgia Kouyialis and Ruth Misener. Heuristics with performance guarantees for the minimum number of matches problem in heat recovery network design. *Computers & Chemical Engineering*, **2018**. arXiv v2 (11 Apr 2018) read; data, code and solver logs at github.com/dimletsios/min_matches_heuristics.
DOI: 10.1016/j.compchemeng.2018.03.002 arXiv: 1709.04688 1, 2, 3, 8

[2] Soterios A. Papoulias and Ignacio E. Grossmann. A structural optimization approach in process synthesis—II: Heat recovery networks. *Computers & Chemical Engineering*, **1983**.
DOI: 10.1016/0098-1354(83)85023-6 3

[3] Kevin K. H. Cheung, Ambros Gleixner and Daniel E. Steffy. Verifying integer programming results. In: *Integer Programming and Combinatorial Optimization (IPCO 2017)*. **2017**.
DOI: 10.1007/978-3-319-59250-3_13 6

[4] The VIPR developers. VIPR: verified integer programming results, certificate format 1.1 and tools. Source code, github.com/scipopt/vipr, commit 30f2951 (29 Oct 2025); specification cert_spec_v1_1.md. **2025**. 6

[5] The SCIP Optimization Suite developers. SCIP 10.0.2 and SoPlex 8.0.2. Source code, github.com/scipopt/scip (tag v10.0.2, commit b8eaf98) and github.com/scipopt/soplex (tag v8.0.2, commit cfadddc). **2026**. 6

[6] Leon Eifler and Ambros Gleixner. A computational status update for exact rational mixed integer programming. *Mathematical Programming*, **2023**.
DOI: 10.1007/s10107-021-01749-5 6

[7] Georgia Kouyialis. Symmetry and degeneracy in nonconvex optimisation problems: Application to heat recovery networks. PhD thesis. Imperial College London, **2019**.
DOI: 10.25560/67897 8

[8] Dimitrios Letsios, Radu Baltean-Lugojan, Francesco Ceccon, Miten Mistry, Johannes Wiebe and Ruth Misener. Approximation algorithms for process systems engineering. *Computers & Chemical Engineering*, **2020**.
DOI: 10.1016/j.compchemeng.2019.106599 8

[9] Kevin C. Furman and Nikolaos V. Sahinidis. Approximation algorithms for the minimum number of matches problem in heat exchanger network synthesis. *Industrial & Engineering Chemistry Research*, **2004**. Read in full; Table 3.
DOI: 10.1021/ie0499781 8

[10] Jaime Cerdá, Vanina G. Cafaro and Diego C. Cafaro. Novel mathematical approaches for the structural synthesis of heat exchanger networks. *Industrial & Engineering Chemistry Research*, **2022**. Read in full (main text; Supporting Information not read).
DOI: 10.1021/acs.iecr.1c02372 8

[11] Yang Chen, Ignacio E. Grossmann and David C. Miller. Computational strategies for large-scale MILP transshipment models for heat exchanger network synthesis. *Computers & Chemical Engineering*, **2015**. Origin of the balanced/unbalanced instances; full text not read.
DOI: 10.1016/j.compchemeng.2015.05.015 8